MODULE 5

NLP & Sequence Models

L1: Seq2Seq & Embeddings

L2: RNNs & BPTT

L3: LSTMs & GRUs

L4: Seq2Seq & Attention

Introduction to Deep Learning

4 Lectures · 50 min each · Week 5

LECTURE 1 · MODULE 5

Seq2Seq Tasks & Word Embeddings

1

- NLP tasks and why sequences differ from fixed-size inputs
- From one-hot encoding to dense vector representations
- Word2Vec Skip-gram: objective and full derivation
- Negative sampling (SGNS) and the noise distribution
- Vector space geometry and semantic analogies

NLP: Task Landscape

Sentiment Analysis	Machine Translation	NER	Language Modelling	Question Answering	Summarisation
Seq → Label	Seq → Seq	Seq → Seq	Seq → next token	(Q,Doc) → Span	Long → Short seq
"Great film!" → Positive	"Hello" → "Hola"	"Apple Inc." → ORG	"The cat sat" → "on"	"Who?" + doc → answer	article → headline

Variable-length sequences

- Input and/or output length is not fixed — unlike images (H×W×C)
- Tokens are discrete symbols — not natural numbers
- Context matters: 'bank' near 'river' ≠ 'bank' near 'money'
- Order matters: 'dog bites man' ≠ 'man bites dog'

Why Standard Architectures Fail

- MLP: fixed input/output size; destroys sequential structure
- CNN: captures local patterns but no notion of long-range order
- Both require a fundamentally different inductive bias: recurrence
- We need to process tokens one at a time, maintaining state

From One-Hot Vectors to Dense Embeddings

One-Hot Encoding

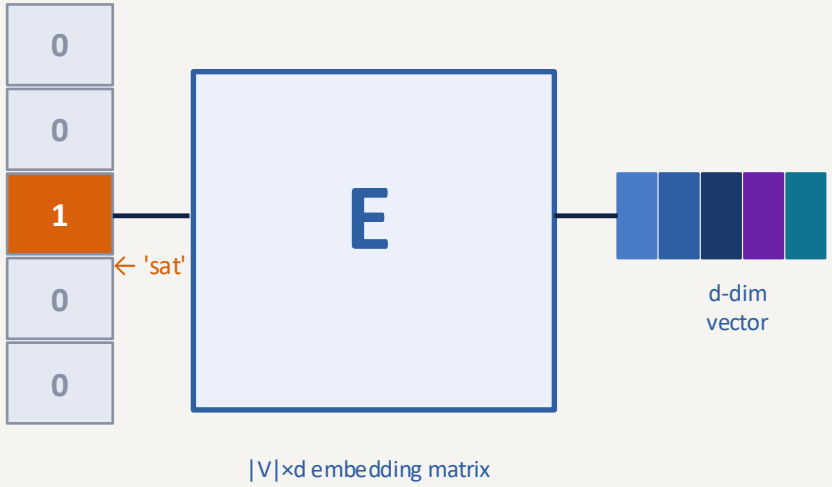
- Vocabulary V = {"the","cat","sat","on","mat"}, |V|=5
- "cat" → [0, 1, 0, 0, 0]
- Dimension = |V| — up to 50,000 in practice
- Dot product of any two one-hot vectors = 0 (all orthogonal)
- No similarity: cat·dog = 0, same as cat·airplane = 0

Dense Embeddings

- Map each word index to a low-dimensional real vector
- Embeddings are learned — similar words get similar vectors
- One-hot × E = lookup of row w in E
- Parameters: |V|×d (e.g. 50K × 300 = 15M)

$$e = E[w], \quad E \in \mathbb{R}^{|V| \times d}$$

Embedding lookup:



Word2Vec: Skip-gram Architecture

Core Idea (Mikolov et al., 2013)

- Given a centre word, predict surrounding context words
- Hypothesis: words with similar contexts have similar meanings
- Context window size k: words within ±k positions
- Two embedding matrices: W (input) and W' (output/context)

$$J = \frac{1}{T} \sum_{t} \sum_{-k \le j \le k,\ j \ne 0} \log P(w_{t+j} | w_t)$$

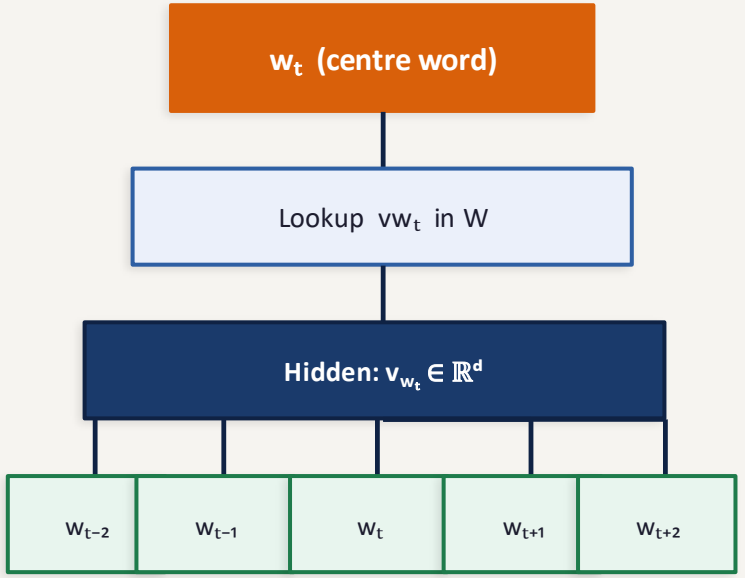
Why two embedding matrices?

- If W = W': self-score = $||v_w||^2$ — always highest for any word
- Asymmetry is necessary to break trivial self-prediction

After training

- W is kept as word embeddings; W' is discarded
- Some implementations average W and W' (double embeddings)

Skip-gram: one centre word → C context words



context words (window k=2)

$$P(o|c) = \frac{\exp(\tilde{v}_o^\top v_c)}{\sum_{w \in V} \exp(\tilde{v}_w^\top v_c)}$$

Skip-gram: Probability Model & Derivation

Deriving P(context | centre):

Step 1 — Similarity score: dot product of output and input vectors

$$\text{score}(o, c) = \tilde{v}_o^\top \cdot v_c$$

$v^c \in \mathbb{R}^d$ from input matrix W; $\tilde{v}_o \in \mathbb{R}^d$ from output matrix W'.

Step 2 — Softmax over all V words gives probability distribution

$$P(o \mid c) = \frac{\exp(\tilde{v}_o^\top v_c)}{\sum_{w \in V} \exp(\tilde{v}_w^\top v_c)}$$

Denominator sums over ALL words in vocabulary — expensive for large |V|!

Step 3 — Negative log-likelihood loss for one (centre, context) pair

$$J(o, c) = -\log P(o|c) = -\tilde{v}_o^\top v_c + \log \sum_{w \in V} \exp(\tilde{v}_w^\top v_c)$$

Minimising J pushes the correct pair score up and all other pairs down.

Step 4 — Gradient w.r.t. centre vector v^c

$$\frac{\partial J}{\partial v_c} = -\tilde{v}_o + \mathbb{E}_{w \sim P}[\tilde{v}_w]$$

Gradient = −(true context vector) + (expected context vector under current model). Push v^c toward $\tilde{v}_o$, away from all others.

Negative Sampling (SGNS)

The Problem with Full Softmax

- $\Sigma_{w \in V} \exp(\tilde{v}_w^T v_c)$ requires $|V|$ dot products per step
- With $|V|$=50K and d=300: millions of multiplications per gradient step
- Solution: replace the 50K-way classifier with a binary one

SGNS: Binary Classifier

- Is (centre, context) pair real or noise?
- Sample k negatives from $P_n(w) \propto \text{freq}(w)^{(3/4)}$
- k = 5–20 in practice — thousands of times cheaper
- Both W and W' are still fully learned via backprop

SGNS Objective:

$$J = \log \sigma(\tilde{v}_o^\top v_c) + \sum_{i=1}^{k} \mathbb{E}[\log \sigma(-\tilde{v}_{w_i}^\top v_c)]$$

Noise distribution:

$$P_n(w) \propto \text{freq}(w)^{3/4}$$

Gradient (only k+1 dot products)

- Push v^c toward true context $\tilde{v}_o$
- Push v^c away from k noise words $\tilde{v}_i$

$$\frac{\partial J}{\partial v_c} = (1 - \sigma(\tilde{v}_o^\top v_c))\tilde{v}_o - \sum_{i=1}^{k} \sigma(\tilde{v}_{w_i}^\top v_c)\tilde{v}_{w_i}$$

	Full Softmax	SGNS
Dot products	$\|V\| \approx 50K$	$k+1 \leq 21$
W' rows updated	All $\|V\|$	k+1 only
Speedup	—	**~2,500×**

Word Vectors: Geometric Properties

The Analogy Test

- Vector arithmetic captures semantic relationships:
- Emerged purely from co-occurrence statistics

$$\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} + \mathbf{v}_{\text{woman}} \approx \mathbf{v}_{\text{queen}}$$

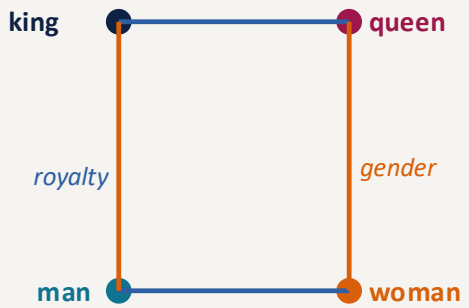
Cosine Similarity:

$$\text{sim}(u, v) = \frac{u^\top v}{\|u\| \cdot \|v\|} \in [-1, 1]$$

Known Limitations

- Polysemy: 'bank' has one vector for multiple meanings
- Static: same vector regardless of context
- Fix: contextualised embeddings (BERT, GPT) — Module 6

Vector space (2D projection):



LECTURE 2 · MODULE 5

RNNs & Backprop Through Time

2

- RNN architecture: the recurrence equation
- Forward pass — all 4 equations
- BPTT: full gradient derivation
- Vanishing & exploding gradients: mathematical analysis
- Gradient clipping

Recurrent Neural Network: Core Architecture

The Recurrence Equations:

$$h_t = \tanh(W_{hh}\, h_{t-1} + W_{xh}\, x_t + b_h)$$

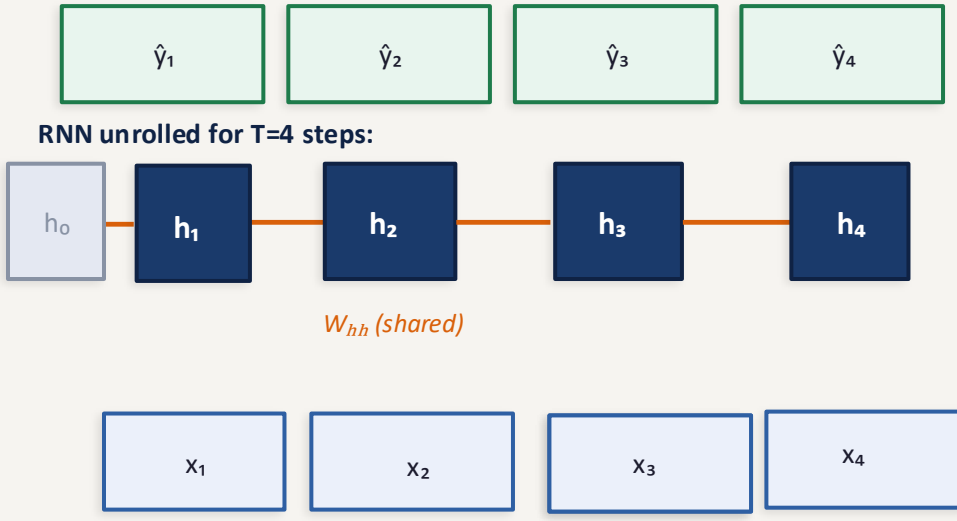
$$\hat{y}_t = W_{hy}\, h_t + b_y$$

Parameter count

- W_{hh}: H×H W_{xh}: H×d b_h: H W_{hy}: C×H b_y: C
- Total: H(H+d+1) + C(H+1) — independent of sequence length T!
- Same matrices applied at every timestep (weight sharing)

Initialisation

- $h_0 = 0$ (zero vector, or learned parameter)
- First hidden state has no history — computed from x_1 only



RNN Forward Pass: Full Equations

Complete forward pass for $x_1, x_2, \ldots, x_T$:

Pre-activation — linear combination of previous state and current input

$$z_t = W_{hh}\, h_{t-1} + W_{xh}\, x_t + b_h$$

$z_t \in \mathbb{R}^H$. Two matrix-vector products: H×H and H×d. Additive bias b_h.

Hidden state — non-linear activation applied element-wise

$$h_t = \tanh(z_t) \in \mathbb{R}^H$$

tanh maps $\mathbb{R} \to (-1,1)$. Saturates for large $|z|$ — a source of vanishing gradients.

Output logits — only computed at steps where prediction is needed

$$o_t = W_{hy}\, h_t + b_y \in \mathbb{R}^C$$

C = vocabulary size for language modelling. Apply softmax to get $\hat{y}_t = \text{softmax}(o_t)$.

Cross-entropy loss at step t

$$L_t = -\log \hat{y}_t[y_t] = -o_t[y_t] + \log \sum_j \exp(o_t[j])$$

y_t is the ground-truth index. Total loss: $L = (1/T)\ \Sigma_t\, L_t$ (average over timesteps).

Backpropagation Through Time (BPTT): Derivation

Goal: compute $\partial L/\partial W_{hh}$ where $L = \Sigma_t L_t$ and h_t depends on W_{hh} at every step

Step 1 — Loss gradient w.r.t. output logits (standard cross-entropy)

$$\partial L_t/\partial o_t = \hat{y}_t - e_{y_t}$$

e_{yt} is the one-hot for true class y_t. Flows back to h_t via W_{hy}.

Step 2 — Gradient w.r.t. h_t (receives gradient from L_t and h_{t+1})

$$\delta h_t = W_{hy}^{\top}(\hat{y}_t - e_{y_t}) + W_{hh}^{\top}(\delta h_{t+1} \odot (1 - h_{t+1}^2))$$

Two sources: output loss at t, and backprop from the NEXT state via W_{hh}.

Step 3 — Through tanh: multiply by local Jacobian diag($1 - h_t^2$)

$$\delta z_t = \delta h_t \odot (1 - h_t^2)$$

If $|h_t| \approx 1$ (saturated), then $1 - h_t^2 \approx 0 \rightarrow$ gradient signal is killed.

Step 4 — Accumulate gradient for W_{hh} over all timesteps

$$\frac{\partial L}{\partial W_{hh}} = \sum_t \delta z_t \cdot h_{t-1}^{\top}$$

W_{hh} is shared across time so gradients from ALL steps are SUMMED.

Vanishing & Exploding Gradients

Key quantity: gradient of h_t w.r.t. h_{t-k} (k steps back)

$$\frac{\partial h_t}{\partial h_{t-k}} = \prod_{j=t-k+1}^{t} \text{diag}(1 - h_j^2) \cdot W_{hh}$$

Vanishing Gradient

$$\left\| \frac{\partial h_t}{\partial h_{t-k}} \right\| \leq (\lambda_{\max} \cdot \gamma)^k$$

As k → ∞, (λ·γ)^k → 0 if λ·γ < 1
Gradient vanishes → no long-range learning

Exploding Gradient

- If eigenvalues of $W_{hh} > 1$: gradient norm grows exponentially
- Fix: gradient clipping — preserve direction, cap magnitude

$$\text{if } \|g\| > \tau: \; g \leftarrow g \cdot \frac{\tau}{\|g\|}$$

Gradient Clipping

- if ||g|| > threshold:
- g ← g × threshold / ||g||
- Typical threshold: 1.0 or 5.0

Why Vanishing Is the Real Problem

- Exploding: easy to detect (NaN/Inf), easy to fix (clipping)
- Vanishing: silent failure — loss decreases but long-range patterns aren't learned
- Root cause: repeated multiplication by W_{hh} across many steps
- Fix: LSTM/GRU gates control information flow → Lecture 3

LECTURE 3 · MODULE 5

Gated Units: LSTMs & GRUs

3

- LSTM: cell state, forget gate, input gate, output gate
- All 6 LSTM equations derived
- Why the cell state creates a gradient highway
- GRU: reset gate, update gate — 4 equations
- LSTM vs GRU comparison

LSTM: Motivation & the Gradient Highway

What we want

- Remember information across many timesteps without gradient decay
- Forget irrelevant past information adaptively
- Selectively write new information to memory
- All controlled by the network itself

Cell State — Constant Error Carousel

- Hochreiter & Schmidhuber (1997): introduce a CELL STATE c_t
- c_t flows through time with only $\odot$ and + — no matrix multiply!
- Gates modulate what enters and exits the cell state

Key gradient identity:

$$\frac{\partial c_t}{\partial c_{t-1}} = f_t \quad \text{(not } W_{hh}\text{!)}$$

If $f_t \approx 1$: gradient × 1 at every step — no decay!

RNN vs LSTM gradient path:

Plain RNN:

Gradient × W_{hh} at every step → exponential decay

LSTM cell state:

Gradient flows through additions — no decay!

Why Addition Fixes Vanishing

- $\partial c_t/\partial c_{t-1} = f_t$ (element-wise, not a matrix product)
- If $f_t \approx 1$ over many steps: gradient magnitude ≈ 1^k = 1
- No repeated matrix multiplication → no exponential decay

LSTM: All 6 Equations

Inputs: $x_t \in \mathbb{R}^d$, $h_{t-1} \in \mathbb{R}^H$, $c_{t-1} \in \mathbb{R}^H$

Forget gate f_t — decides what to erase from cell state

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \in (0,1)^H$$

$f_t \approx 0$: forget c_{t-1}. $f_t \approx 1$: keep everything. $[h_{t-1}, x_t]$ = concatenation.

Input gate i_t — decides which new values to store

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \in (0,1)^H$$

Controls how much of candidate $\hat{c}_t$ gets written. $i_t \approx 0$: ignore. $i_t \approx 1$: fully write.

Candidate cell $\hat{c}_t$ — proposed new content (tanh like vanilla RNN)

$$\hat{c}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \in (-1,1)^H$$

Proposes information from current input and previous hidden state. Gated by i_t.

Cell state update — forget old + write new (the gradient highway)

$$c_t = f_t \odot c_{t-1} + i_t \odot \hat{c}_t$$

No weight matrix — only additions and $\odot$. Derivative w.r.t. c_{t-1} is just f_t.

Output gate o_t — decides how much cell state to expose

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \in (0,1)^H$$

Even if cell state carries important info, network can choose not to expose it.

Hidden state h_t — filtered view of cell state

$$h_t = o_t \odot \tanh(c_t)$$

Passed to next layer and next timestep. tanh squashes c_t values to (−1,1).

GRU: Gated Recurrent Unit

GRU Motivation (Cho et al., 2014)

- LSTM has 4 parameter groups — expensive to train
- GRU merges cell state and hidden state into one vector
- Only 2 gates → 3 parameter groups instead of 4
- Often matches LSTM performance with less compute

GRU Equations — all 4 steps:

Reset gate r_t — how much to forget previous hidden state

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t] + b_r) \in (0,1)^H$$

$r_t \approx 0$: ignore past h_{t-1} when computing candidate (fresh start). $r_t \approx 1$: full access.

Update gate z_t — interpolation between past and candidate

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t] + b_z) \in (0,1)^H$$

A single gate does the work of both f and i in LSTM.

Candidate hidden state $\tilde{h}_t$

$$\tilde{h}_t = \tanh(W_h \cdot [r_t \odot h_{t-1},\ x_t] + b_h)$$

Reset gate r_t filters h_{t-1} before computing candidate. If $r_t=0$, $\tilde{h}_t$ depends only on x_t.

Hidden state — linear interpolation controlled by update gate

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

$z_t=0$: copy old state. $z_t=1$: fully replace with candidate. No separate cell state.

LECTURE 4 · MODULE 5

Seq2Seq & Attention Mechanisms

4

- Encoder-decoder architecture
- The information bottleneck and why it fails
- Bahdanau (additive) attention — full derivation
- Luong (multiplicative) attention
- Scaled dot-product attention

Seq2Seq: Encoder-Decoder Architecture

The Seq2Seq Task

- Map variable-length source to variable-length target
- E.g. machine translation, summarisation

Encoder:

$$h_t^e = \text{LSTM}(h_{t-1}^e, x_t)$$

Final hidden state becomes context vector c = h_Tx

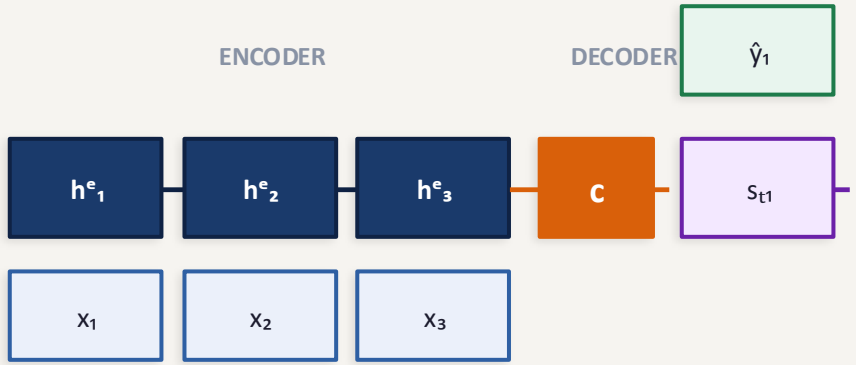
Encodes the ENTIRE source into one fixed-size vector — the bottleneck!

Decoder:

$$s_t = \text{LSTM}(s_{t-1}, [y_{t-1}; c_t])$$

$$P(y_t | y_{<t}, x) = \text{softmax}(W_s s_t + b_s)$$

Encoder-Decoder diagram:



Bahdanau Attention: Full Derivation

Given: encoder states $h_1^e,\ldots,hT_x^e$ and decoder state at step t: s_{t-1}

Step 1 — Alignment score e_{tj}: how relevant is encoder position j for decoder step t?

$$e_{tj} = v_a^\top \tanh(W_a s_{t-1} + U_a h_j^e)$$

$W_a \in \mathbb{R}^{A\times H}$, $U_a \in \mathbb{R}^{A\times H}$, $v_a \in \mathbb{R}^A$ are learned. Both s_{t-1} and h_j^e mapped to common space via tanh.

Step 2 — Softmax over scores → attention weights (soft distribution over source)

$$\alpha_{tj} = \frac{\exp(e_{tj})}{\sum_{j'} \exp(e_{tj'})}$$

$\alpha_{tj} \in (0,1)$ and $\Sigma_j \alpha_{tj} = 1$. Probability that decoder step t attends to encoder position j.

Step 3 — Context vector c_t: weighted sum of encoder hidden states

$$c_t = \sum_j \alpha_{tj} h_j^e$$

Different c_t at every decoder step! Unlike basic Seq2Seq which uses one fixed c. Dynamic context.

Step 4 — Decoder update using dynamic context c_t

$$s_t = \text{LSTM}(s_{t-1}, [y_{t-1}; c_t])$$

c_t concatenated with previous token y_{t-1} as decoder input. Direct access to relevant source parts.

Scaled Dot-Product Attention & Score Function Comparison

Vaswani et al. (2017) — Attention Is All You Need:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

Why divide by $\sqrt{d_k}$?

$$q, k \sim \mathcal{N}(0, 1) \Rightarrow q^\top k \sim \mathcal{N}(0, d_k)$$

Large d_k → large dot products → softmax saturation → tiny gradients → divide by $\sqrt{d_k}$ keeps variance ≈ 1

Attention Gradient — No Bottleneck

$$\frac{\partial L}{\partial h_j^e} = \sum_t \alpha_{tj} \cdot \frac{\partial L}{\partial c_t}$$

Every encoder state h_j^e gets gradient from ALL decoder steps (weighted by α_{tj})

Name	Score function	Params
Additive (Bahdanau)	$v_a^T \tanh(W_a s + U_a h)$	W_a, U_a, v_a
Dot-product (Luong)	$s^T h$	none
General (Luong)	$s^T W_a h$	W_a only
Scaled dot (Transformer)	$q^T k / \sqrt{d_k}$	Wq, Wk, Wv

One question, many answers

All score functions ask: how similar is this query to this key?

Module 5 Summary

Summary

L1 — Embeddings

- One-hot → dense; Skip-gram SGNS objective
- $P(o|c) = \text{softmax}(\tilde{v}_o^T v^c)$; vector arithmetic = semantics

$$J = \log \sigma(\tilde{v}_o^\top v_c) + \sum_{i=1}^{k} \mathbb{E}[\log \sigma(-\tilde{v}_{w_i}^\top v_c)]$$

L2 — RNNs & BPTT

- $h_t = \tanh(W_{hh}h_{t-1}+W_{xh}x_t+b_h)$; shared weights across time
- $\partial L/\partial W_{hh} = \Sigma_t \delta z_t h_{t-1}^T$; $(\lambda\cdot\gamma)^k \to 0$ for vanishing

$$\frac{\partial h_t}{\partial h_{t-k}} = \prod_{j=t-k+1}^{t} \text{diag}(1 - h_j^2) \cdot W_{hh}$$

L3 — LSTMs & GRUs

- LSTM: 6 equations; $\partial c_t/\partial c_{t-1} = f_t$ (gradient highway)
- GRU: 4 equations; 25% fewer params; both fix vanishing

$$c_t = f_t \odot c_{t-1} + i_t \odot \hat{c}_t$$

L4 — Seq2Seq & Attention

- Encoder-decoder with fixed context $c = hT_x^e$
- Bahdanau: $e_{tj}=v_a^T\tanh(W_a s_{t-1}+U_a h_j)$; $c_t=\Sigma\alpha_{tj}h_j$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$